

Parameter Passing

Computer Engineering 1

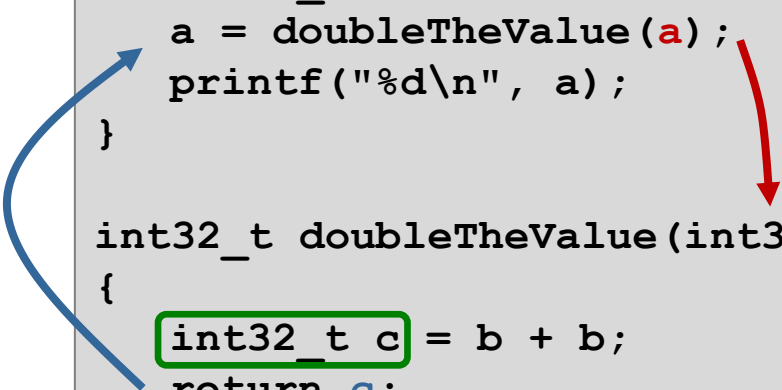
Motivation

```
#include <stdio.h>

int32_t doubleTheValue(int32_t b);

int32_t main(void)
{
    int32_t a = 5;
    a = doubleTheValue(a);
    printf("%d\n", a);
}

int32_t doubleTheValue(int32_t b)
{
    int32_t c = b + b;
    return c;
}
```



The diagram illustrates the flow of data between the `main` function and the `doubleTheValue` function. A blue arrow originates from the `a` argument in the function call `a = doubleTheValue(a);` inside `main` and points to the `int32_t c` variable in the `doubleTheValue` function, indicating that the value of `a` is passed to `c`. A red arrow originates from the `c` variable in the `doubleTheValue` function and points back to the `a` variable in the `main` function, indicating that the value of `c` is returned to `a`.

- How does `main()` pass the value of `a` to the function?
- Where is the local variable `c` stored?
- How is the value of `c` returned to `main()`?

Agenda

- **Parameter Passing**
- **Passing through Registers**
- **Passing through Global Variables**
- **Reentrancy**
- **ARM Procedure Call Standard**
- **Functions - Stack Frame**
- **Calling Assembly Subroutines from C**

Learning Objectives

At the end of this lesson you will be able

- to explain and classify the different possibilities to pass data between different parts of the program
- to outline what an Application Binary Interface is
- to name the roles of the different registers in the ARM Procedure Call Standard
- to enumerate and describe the operations of the caller of a subroutine
- to summarize the structure of a subroutine and describe what happens in the prolog and epilog respectively
- to explain, interpret and discuss stack frames
- to access elements of a stack frame in assembly
- to understand the build-up and tear-down of stack-frames
- to call an assembly subroutine from a C program

Parameter Passing

- **Where?**

- **Register**

- Caller and Callee¹⁾ use the same register

- **Global variables**

- Shared variables in data area (section)

- **Stack**

- Caller → push parameter on stack
 - Callee → access parameter through `ldr <Rt>, [sp, #<imm>]`

- **How?**

- **pass by value**

- Handover the value

- **pass by reference**

- Handover the address to a value

¹⁾ Caller: the routine that calls the subroutine; Callee: the subroutine being called

Passing through Registers

"pass by value"

- Values in agreed registers, e.g.
 - **r1** Parameter: Caller → function
 - **r0** Return value of function
- Efficient and simple
- Limited number of registers
 - How do we pass tables and structs?

	<pre>.datatext ... movs r1, #0x03 bl double movs ..., r0 ...</pre>
<i>caller</i>	
<i>callee</i>	<pre>double: lsls r0, r1, #1 bx lr</pre>
<i>function</i>	
<i>double</i>	

Passing through Registers

"pass by reference"

- Pass reference (= address) of data structure in register
- Allows passing of larger structures
- Example
 - Function doubleTableValues
 - doubles each value in the table
 - **r0** Caller passes address of p1Table
 - **r1** Caller passes length of table (pass by value)

```
TLENGTH = 16

.data
p1Table:
.skip TLENGTH

.text
... 1)
ldr    r0,=p1Table
movs   r1,#TLENGTH
bl     doubleTableValues
...

doubleTableValues:
movs   r2,#0
loop:
ldrb   r4,[r0,r2]
lsls   r4,r4,#1
strb   r4,[r0,r2]
adds   r2,#1
cmp    r2,r1
blo    loop
bx     lr
```

caller

callee
function
doubleTableValues

Passing through Global Variables

- Shared variables in data area
 - **param1** Caller → procedure
 - **result** Return value
- Overhead to access variable
 - In Caller and Callee
- Error-prone, unmaintainable
 - No encapsulation
 - Many dependencies
 - Multiple use of the same variable
 - Where is the variable written?
 - Who is allowed to read variable?
 - Requires unique variable names
 - Challenge if there is a large number of modules

caller

callee
function
double_g

```
.data
param1: .skip 1
result: .skip 1

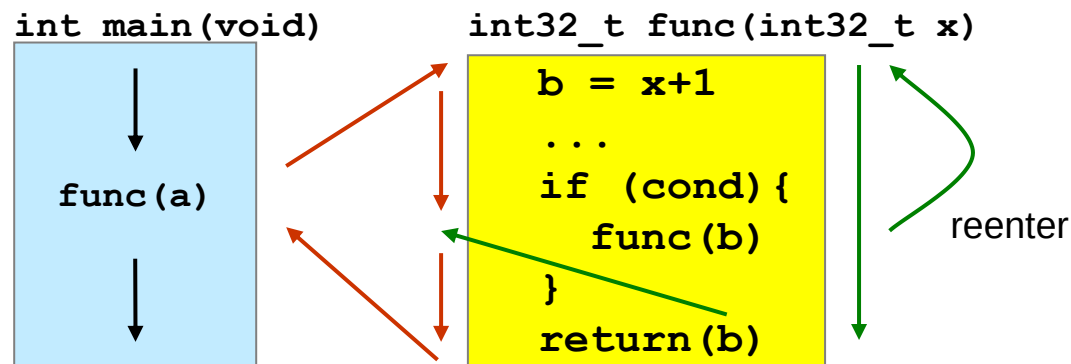
.text
...
ldr      r4,=param1
movs     r5,#0x03
strb     r5,[r4]
bl       double_g
ldr      r4,=result
ldrb     ..., [r4]
...

double_g:
ldr      r4,=param1
ldrb     r1,[r4]
lsls     r0,r1,#1
ldr      r4,=result
strb     r0,[r4]
bx       lr
```


Reentrancy

– Recursive Function Calls?

- Registers and global variables are overwritten
- Requires an own set of data for each call
 - Parameters / Local variables



requires a red **b**
and a green **b**

– Solution

- Combined use of registers and stack for parameter passing
- See ARM Procedure Call Standard

ARM Procedure Call Standard

- **Procedure Call Standard for the ARM architecture (AApcS)**

- <https://github.com/ARM-software/abi-aa/releases>

- Part of the ABI (Application Binary Interface) for the ARM Architecture

- ABI → Specification to which independently produced relocatable object files must conform to be statically linkable¹⁾ and executable

- Function calls

- Parameter passing

- Binary formats of information



source: colourbox

“The AAPcS defines how subroutines can be separately written, separately compiled, and separately assembled to work together. It describes a contract between a calling routine (caller) and a called routine (callee).”

→ Enables interaction of code produced by different compilers.

¹⁾ Linking will be covered later

ARM Procedure Call Standard

- **AApCS specifies**

- Most of the items have already been covered in this course

Layout of data	• Size, alignment, layout of fundamental data types
Register Usage	• What are the registers used for
Memory Sections and Stack	• Code, read-only data, read-write data, stack, heap
Stack	• Full-descending, word-aligned, ...
Subroutine Calls	• Mechanism using lr and pc
Result Return	• Returning arguments through r0 (and r1 – r3)
Parameter Passing	• Passing arguments in r0-r3 and on stack

ARM Procedure Call Standard

– Register Usage

Register	Synonym	Role
r0	a1	Argument / result / scratch register 1
r1	a2	Argument / result / scratch register 2
r2	a3	Argument / scratch register 3
r3	a4	Argument / scratch register 4
r4	v1	Variable register 1
r5	v2	Variable register 2
r6	v3	Variable register 3
r7	v4	Variable register 4
r8	v5	Variable register 5
r9	v6	Variable register 6
r10	v7	Variable register 7
r11	v8	Variable register 8
r12	ip	Intra-Procedure-call scratch register ¹⁾
r13	sp	
r14	lr	
r15	pc	

Register contents
might be modified
by callee

Callee must
preserve contents
of these registers
(Callee saved)

Cortex-M0: Registers
r8 – r11 have limited set of
instructions. Therefore,
they are often not used by
compilers.

¹⁾ used by the linker

ARM Procedure Call Standard

- **Scratch Register**
 - Used to hold an intermediate value during a calculation
 - Usually, such values are not named in the program source and have a limited lifetime
- **Variable Register**
 - A register used to hold the value of a variable, usually one local to a routine, and often named in the source code.
 - Cortex-M0 registers r8 – r11 (v5 – v8) are often unused
 - as they are accessible only by few instructions
- **Argument, Parameter**
 - Used interchangeably
 - Formal parameter of a subroutine

ARM Procedure Call Standard

- **Parameters**
 - Caller copies arguments to r0 to r3
 - Caller copies additional parameters to stack
- **Returning fundamental data types**
 - Smaller than word zero or sign extend to word; return in r0
 - Word return in r0
 - Double-word return in r0 / r1 ¹⁾
 - 128-bit return in r0 – r3 ¹⁾
- **Returning composite data types (*structs, arrays, ...*)**
 - Up to 4 bytes return in r0
 - Larger than 4 bytes stored in data area; address passed as extra argument at function call

¹⁾ Least significant word stored in lowest register

ARM Procedure Call Standard

– Example

```
void caller(void)
{
    uint32_t p = 4;
    uint32_t q = 5;
    uint32_t r = 6;
    uint32_t sum;

    sum = callee(p,q,r);
}
```

```

movs    r4,#4
movs    r5,#5
movs    r6,#6
} local variables
  r4 – r6

mov     r2,r6
mov     r1,r5
mov     r0,r4
} copy parameters
  to r0 – r2

bl      callee

mov     r7,r0
} copy return value
  to local variable
```

```
uint32_t callee(uint32_t a,
                uint32_t b,
                uint32_t c)
{
    return a + b + c;
}
```

```

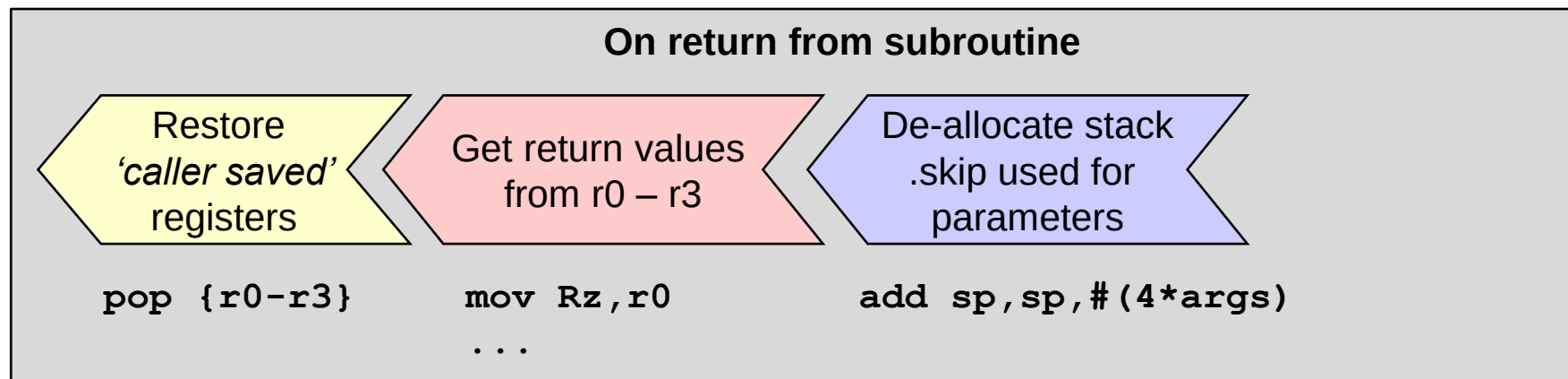
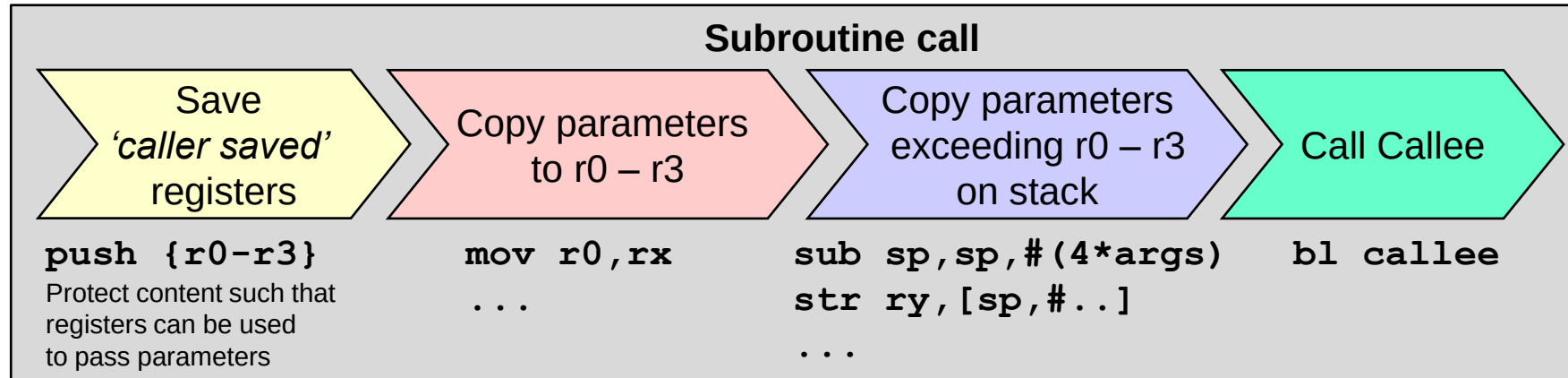
callee:
    adds    r0,r0,r1
    adds    r0,r0,r2
    bx      lr
} callee uses
  own copies
  of parameters
```

push and pop are omitted in the example

ARM Procedure Call Standard

– Subroutine Call – **Caller Side**

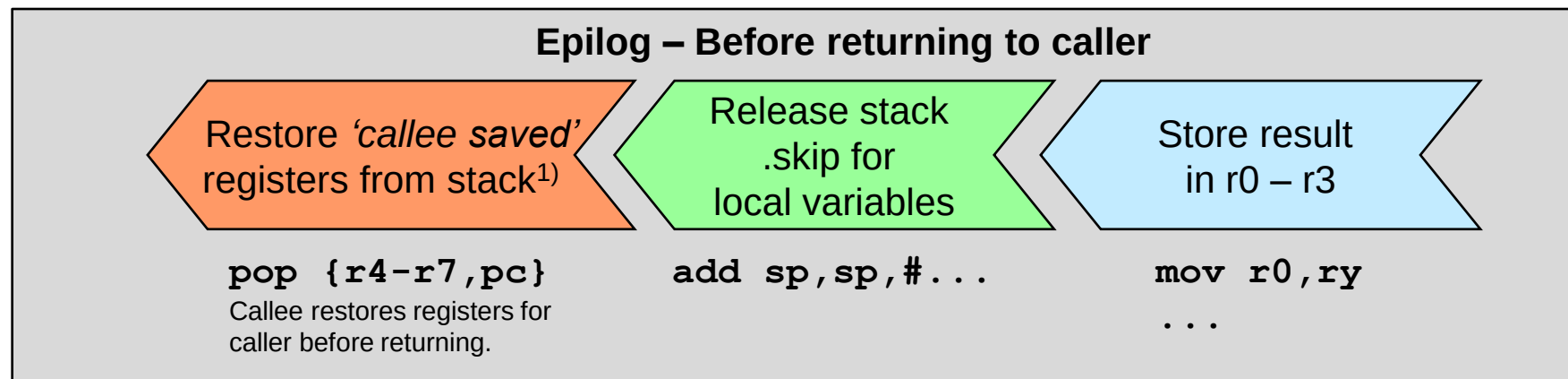
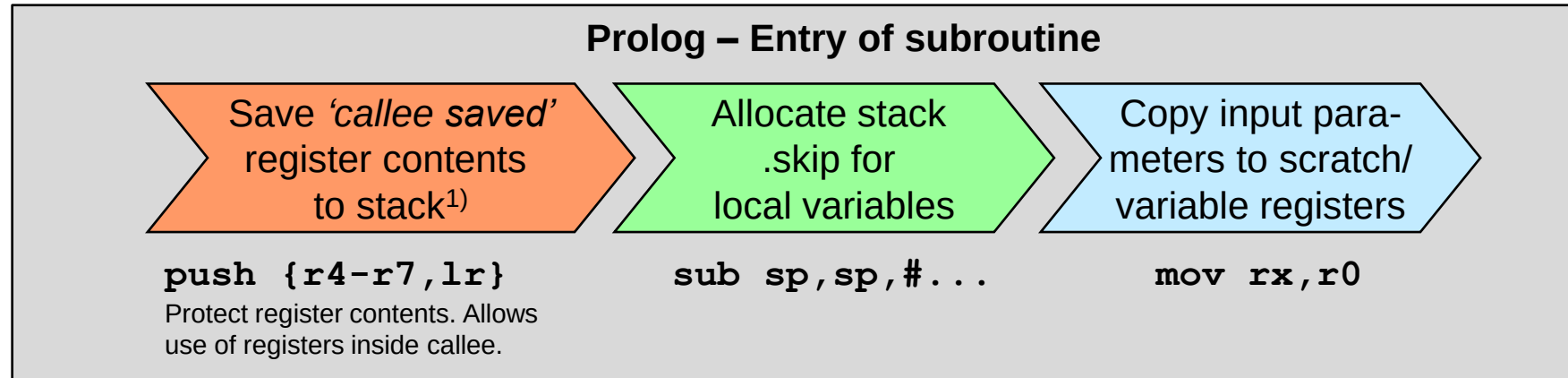
Pattern as used by the compiler. Manually written assembly code may be slightly different.



ARM Procedure Call Standard

– Subroutine Structure – **Callee Side**

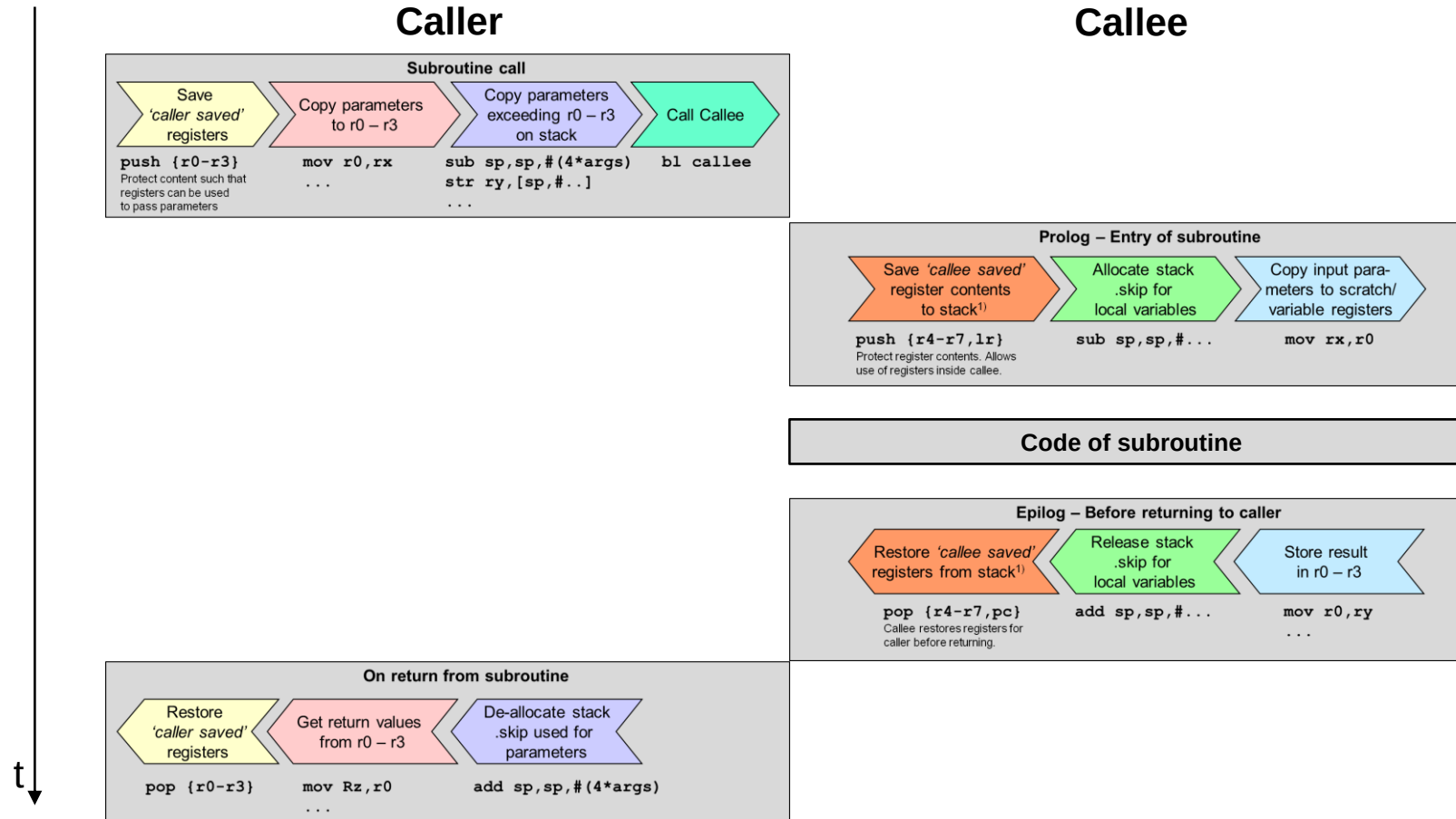
Pattern as used by the compiler. Manually written assembly code may be slightly different.



¹⁾ only registers modified by the callee are saved and restored. r8 – r11 are often unused by the compiler.

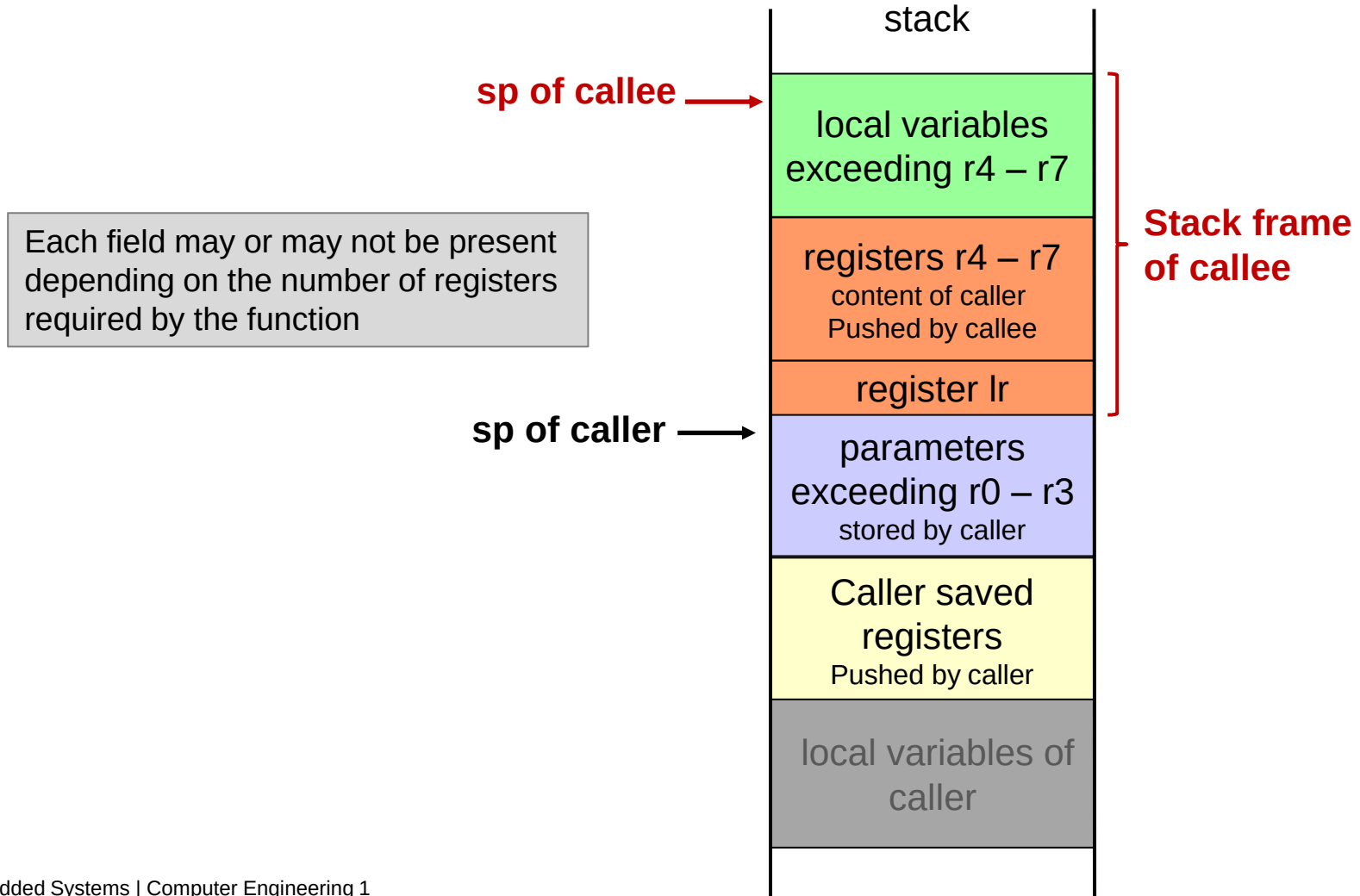
ARM Procedure Call Standard

– Summary – Flow of Execution for a Subroutine Call



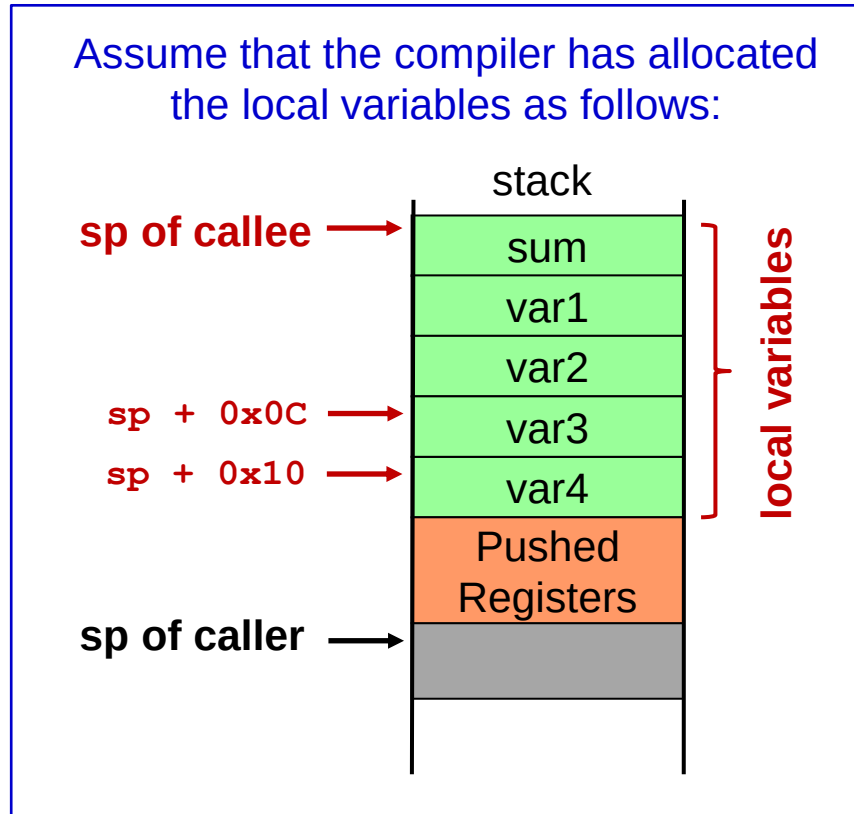
Functions - Stack Frame

– Stack Frame



Functions – Stack Frame

- Access to Local Variables on Stack
 - Example using word sized variables



Compiler uses indirect addressing relative to stack pointer (sp)

```
sum = var3 + var4;  
ldr    r0, [sp, #0x0c]    var3  
ldr    r1, [sp, #0x10]    var4  
adds   r0, r1, r0  
str    r0, [sp, #0x00]    sum
```

r0 and r1 used as scratch register

Functions – Stack Frame

- **Functions in C**
 - Subroutine with return value (usually in register r0 / r1)
 - Function Parameters
 - Always "pass by value"
 - Copy is being passed (not the original)
 - "pass by reference" only possible through use of pointers
 - Pointer itself passed by value
 - Registers r0 – r3
 - starting with the first argument in r0
 - Stack if more space is required
- Local Variables
 - In registers r4 – r7
 - On stack if more space is required or address operator (&) is used

Functions - Stack Frame

– Example

```
int main(void)
{
    uint32_t start = 0x1234;
    uint32_t result = 0;

    result = calc(start);
}

uint32_t calc(uint32_t val)
{
    uint32_t a[8];
    uint32_t res = 0;
    ...
    a[0] = val;
    ...
    return res;
}
```

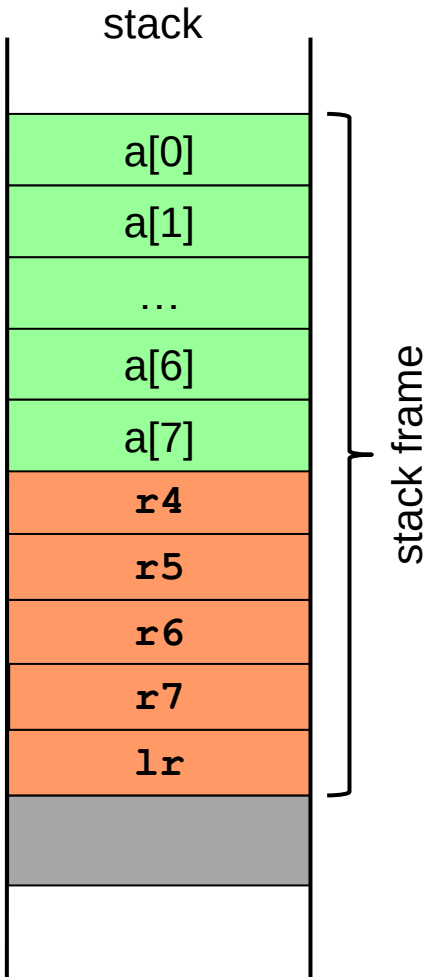
```
mov r0,... @ start
bl  calc

push {r4-r7,lr}
@allocate a[] on stack
sub sp,sp,#0x20

movs r6,#0 ;res
@access a[0]
str r0,[sp,#0x00]

mov r0,r6 ;res
@stack teardown
add sp,sp,#0x20

pop {r4-r7,pc}
```



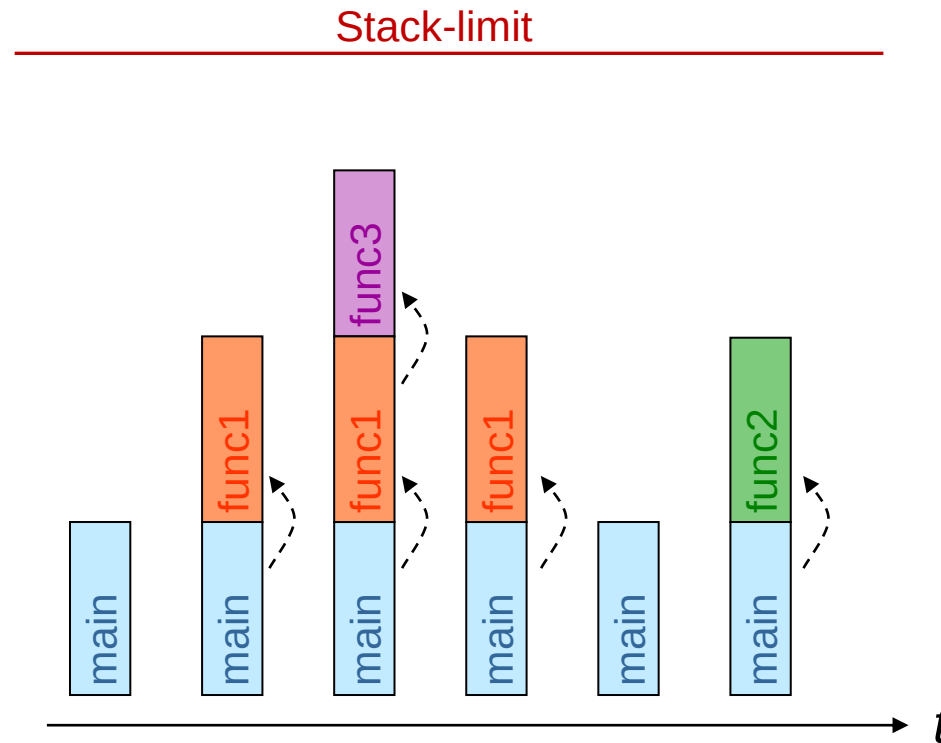
Functions - Stack Frame

- Build-up and tear-down of stack frames

```
int32_t main(void)
{
    int16_t x, y;
    x = func1(...);
    y = func2(...)
}

int16_t func1(...)
{
    z = func3(...)
    ...
}

int16_t func2(...) {...}
int16_t func3(...) {...}
```



Calling Assembly Subroutines from C

```
extern void strcpy(char *d, const char *s);  
int main(void)  
{  
    const char *srcstr = "First string ";  
    char dststr[] = "Second string";  
    strcpy(dststr, srcstr);  
    return (0);  
}
```

```
.text  
.global strcpy  
.type    strcpy, %function  
strcpy:  
    @ r0 points to destination string  
    @ r1 points to source string  
    ldrb r2, [r1]    @ Load byte and update address  
    adds r1, r1, #1  
    strb r2, [r0]    @ Store byte and update address  
    adds r0, r0, #1  
    cmp  r2, #0      @ Check for null terminator  
    bne  strcpy      @ Keep going if not  
    bx   lr          @ Return  
.size strcpy, . - strcpy
```

Example : from ARM Ltd.

Conclusions

- **Parameter Passing**

- Through registers
- Through global variables
- On the stack (stack frame)

- **Procedure Call Standard**

- r0 – r3 parameters / scratch → callee may modify these registers
- r4 – r7 local variables → callee restores potentially modified registers
- r0 – r3 Return

- **Stack Frame**

- Nesting of subroutines
- Stack frame is constructed at function call
- Stack frame is deconstructed when function terminates (returns)

